1 EMBASAMENTO TEÓRICO

Este capítulo apresenta, de forma clara, os conceitos e as tecnologias que fundamentam as atividades descritas neste relatório. Como o estágio teve por foco o desenvolvimento de um módulo de solicitações com a rastreabilidade, a auditoria e reversão para o sistema Banco de Especialistas, é dada ênfase ao *back-end* (Flask, SQLAlchemy e PostgreSQL) e aos conceitos de auditoria e versionamento de dados. As demais tecnologias (Vue.js, Nuxt, Pytest, Git e Docker) são abordadas como elementos de apoio ao desenvolvimento, à validação e à execução do módulo.

1.1 Arquitetura e Padrões de Aplicações Web

1.1.1 Cliente-Servidor

A arquitetura cliente-servidor organiza um sistema distribuído em provedores de serviços e requisitantes (**??**). Nesse contexto, a comunicação orientada pelo estilo REST e pelo protocolo HTTP define interações sem estado, nas quais cada requisição é independente e deve carregar as informações necessárias ao seu processamento (**??**).

1.1.2 REST e HTTP

O estilo arquitetural REST (Representational State Transfer) define um conjunto de restrições que, quando aplicadas, promovem escalabilidade, simplicidade e independência entre cliente e servidor (**??**). A comunicação entre os componentes do sistema ocorre por meio de recursos identificados por URIs, manipulados por métodos HTTP (GET, POST, PUT, DELETE e etc.) e representados em formatos padronizados, como JSON ou XML. A aplicação dessas restrições permite que o cliente e o servidor evoluam de forma independente, desde que a interface entre eles permaneça consistente

1.1.3 MVC e Separação de Responsabilidade (SoC)

O padrão de arquitetura MVC (Model-View-Controller) organiza a aplicação em três componentes: o *Model* representa os dados e as regras de negócio, a *View* é responsável pela apresentação e interação com o usuário, e o *Controller* atua como intermediário, recebendo as requisições da interface, manipulando os dados e retornando respostas (**??**). A separação de responsabilidades (SoC - Separation of Concerns) permite que cada componente evolua independentemente, desde que a interface entre eles permaneça consistente.

Tabela 1 – Códigos HTTP, significados padrão e aplicações no módulo.

Código HTTP	**Significado padrão**	**Aplicação no módulo**
200	Sucesso	Consultas e operações concluídas com êxito.
201	Recurso criado	Criação de uma nova solicitação.
400	Requisição inválida	Erros de validação nos dados enviados.
401	Falha de autenticação	Credenciais inválidas.
403	Proibido	Tentativa de acesso a recurso sem permissão.
404	Não encontrado	Solicitação ou recurso inexistente.
409	Conflito de estado	Tentativa de transição inválida (por exemplo, reverter solicitação não aprovada).

Fonte: Elaboração própria, com base em RFC 9110 (**??**).

1.1.4 Service Layer

O padrão Service Layer organiza a lógica de negócio em uma camada intermediária entre o *Controller* e o *Model*, promovendo a reutilização de regras de negócio e a centralização das operações que envolvem múltiplas entidades (**??**). Essa abordagem facilita a manutenção do sistema, pois as alterações nas regras de negócio podem ser implementadas em um único local, sem impactar diretamente os controladores ou modelos individuais.

1.1.5 Additive Change

O padrão Additive Change descreve a evolução de um sistema por meio da adição de novas funcionalidades, em vez de modificar diretamente o código existente (**??**). Essa abordagem minimiza o risco de introduzir erros em funcionalidades já implementadas e permite que o sistema cresça de forma incremental, preservando a estabilidade e a confiabilidade das operações existentes. No contexto do módulo de solicitações, o Additive Change foi aplicado ao desenvolver novas rotinas para criação, análise e reversão de solicitações, sem alterar as funcionalidades já presentes no Banco de Especialistas.

1.1.6 OWASP e Segurança de Aplicações Web

O Open Worldwide Application Security Project (OWASP) mantém o OWASP Top 10, uma lista de referência global que elenca os dez riscos de segurança mais críticos para aplicações web (**??**). A edição mais recente (2025) destaca que o Controle de Acesso Quebrado (Broken Access Control) permanece no topo do ranking como o risco mais sério, seguido por Configuração Insegura (Security Misconfiguration). Para um sistema que lida com dados de usuários e decisões administrativas, como o Banco de Especialistas, alinhar o desenvolvimento a esse referencial não é apenas uma boa prática, mas uma necessidade para mitigar vulnerabilidades exploráveis.

1.2 Gestão de Projeto Ágil

A organização das atividades de desenvolvimento também exigiu atenção à gestão do projeto, as prioridades e ao acompanhamento do progresso. Em contextos de software, a gestão ágil busca reduzir a incerteza por meio de entregas incrementais, feedback frequente e adaptação contínua às mudanças, em vez de dependência excessiva de planos rígidos e de documentação extensa (**??**). Nesse sentido, a gestão do módulo foi apoiada por ferramentas e práticas que permitiram organizar o backlog, acompanhar pendências e controlar o avanço das tarefas de forma transparente.

1.2.1 OpenProject

O OpenProject é uma plataforma de gestão de projetos e de colaboração que oferece recursos de backlog, planejamento, acompanhamento de tarefas, cronogramas e relatórios de progresso (**??**). No estágio, ele foi utilizado para registrar issues, atribuir responsabilidades, definir prioridades e monitorar o avanço das atividades relacionadas ao desenvolvimento do módulo. Essa ferramenta contribuiu para manter o trabalho organizado, reduzindo a perda de contexto e permitindo uma visão clara do status das pendências em cada etapa do projeto.

1.2.2 Scrum

O Scrum é uma das abordagens ágeis mais conhecidas e organiza o trabalho em ciclos curtos, chamados *sprints*, com papéis definidos, backlog priorizado e cerimônias de planejamento, revisão e retrospectiva (**??**). A estrutura do Scrum é especialmente útil em equipes maiores e em projetos em que a mudança de requisitos é frequente, pois favorece a entrega incremental de valor e a revisão contínua do produto. No caso do estágio, a metodologia não foi seguida em sua forma estrita, mas os princípios de priorização, transparência e acompanhamento incremental foram relevantes para organizar o desenvolvimento do módulo.

1.2.3 Extreme Programming (XP)

O Extreme Programming (XP) enfatiza práticas de engenharia que aumentam a qualidade do software e a capacidade de responder rapidamente às mudanças de requisitos (**??**). Entre suas práticas mais conhecidas estão pequenas entregas, feedback frequente, desenvolvimento orientado a testes, integração contínua, refatoração e foco em simplicidade. Como o módulo foi implementado em um sistema legado, com regras de negócio bem definidas e necessidade de validação constante, o XP se mostrou mais alinhado ao ritmo de trabalho do que a execução rígida de cerimônias do Scrum. As entregas menores, os testes automatizados e a revisão contínua reduziram o risco de regressão e facilitaram a adaptação à estrutura já existente do Banco de Especialistas.

A combinação de priorização em backlog, uso do OpenProject para o acompanhamento das atividades e adoção de práticas do XP permitiu manter um fluxo de trabalho disciplinado,

mesmo em um contexto de desenvolvimento com escopo específico e um único responsável técnico. Essa abordagem equilibrou organização gerencial e rigor técnico, o que foi fundamental para a construção do módulo de solicitações.

1.3 Tecnologias do Back-end

1.3.1 Flask e Blueprints

O Flask foi utilizado na API do módulo por permitir incorporar suas rotas à aplicação existente sem impor uma reorganização do sistema. Os *blueprints* agruparam as operações de criação, análise, acompanhamento e reversão das solicitações em um componente isolado, preservando as demais funcionalidades e facilitando a manutenção do módulo (**??**).

1.3.2 PostgreSQL

O módulo utiliza PostgreSQL para armazenar solicitações, alterações cadastrais e registros de auditoria. A persistência dessas informações precisa manter o cadastro e o histórico coerentes, sobretudo quando uma decisão altera mais de uma entidade.

O PostgreSQL foi adequado por oferecer transações para as operações compostas do fluxo. Na aprovação, a mudança de categoria, a atualização da pontuação, a gravação do *snapshot* do estado anterior e a atualização do status da solicitação devem ser confirmadas em conjunto; se uma etapa falhar, nenhuma alteração deve permanecer aplicada (**??**).

1.3.3 JSONB e Modelagem de Estado

O PostgreSQL oferece o tipo de dado JSONB, que armazena documentos JSON de forma binária, permitindo consultas e indexação eficientes (**??**). O módulo utiliza esse recurso para registrar o estado do especialista no momento da solicitação, preservando a categoria, a pontuação e outros atributos relevantes. Essa abordagem facilita a reversão de alterações e a auditoria das mudanças, mantendo o histórico completo das operações realizadas.

1.3.4 SQLAlchemy e Unidade de Trabalho

O SQLAlchemy foi adotado como ORM para coordenar as alterações de entidades relacionadas em uma única unidade de trabalho (**??**). A sessão acumula as mudanças e, no *commit*, efetiva-as na transação; em caso de falha, o *rollback* descarta o conjunto. Assim, a aprovação não deixa a categoria atualizada sem o *snapshot* correspondente ou sem o novo status da solicitação.

1.3.4.1 Mapeamento Objeto-Relacional e Estratégias de Herança

O SQLAlchemy oferece diferentes estratégias de herança para mapear hierarquias de classes. A estratégia Joined Table Inheritance (herança de tabela unida) cria tabelas separadas

para a classe base e suas subclasses, sendo útil quando há muitos atributos exclusivos. Já a estratégia Single Table Inheritance (herança de tabela única) armazena os diferentes tipos de entidades em uma única tabela, utilizando uma coluna discriminadora. No módulo de solicitações, adotou-se o Single Table Inheritance para os tipos de solicitação (mudança de categoria, recurso e pontuação), pois eles compartilham a maior parte dos atributos, o que evita junções complexas e otimiza as consultas de listagem.

1.3.5 Validação de Dados com Marshmallow e Flasgger

Na fronteira da API, os *schemas* do Marshmallow validam os campos e formatos recebidos antes que os dados cheguem às regras de negócio (**??**). Ao declarar os formatos esperados para as solicitações e suas respostas, também estabelecem o contrato consumido pelas interfaces de criação, acompanhamento e análise do módulo.

1.4 Auditoria, Rastreabilidade e Versionamento

Esta seção reúne os conceitos que sustentam a principal contribuição do módulo: a capacidade de registrar, consultar e desfazer alterações cadastrais com garantias de rastreabilidade.

1.4.1 Auditoria e Rastreabilidade

Auditoria, no contexto de sistemas de informação, é o registro sistemático de quem realizou determinada operação, quando, sobre qual dado e com qual resultado. A rastreabilidade é a propriedade que permite reconstituir o histórico de uma informação a partir desses registros. Fowler (**??**) descreve o padrão *Audit Log* como um registro de alterações feitas nos dados, com o objetivo de permitir verificar o que ocorreu e, quando necessário, compreender a origem de um estado atual. Em sistemas que envolvem decisões de classificação e seleção de pessoas, como o Banco de Especialistas, esse registro contribui para a transparência e para a responsabilização dos agentes envolvidos.

O fluxo anterior ao módulo, baseado em e-mail e execução manual de SQL, ilustra a ausência dessas propriedades: não havia registro formal do solicitante, da justificativa, do responsável pela decisão ou do estado anterior do cadastro. Ao formalizar a solicitação como uma entidade persistente, com status, autoria e datas, o sistema passa a produzir esse registro como efeito natural do próprio fluxo de trabalho.

1.4.2 Fluxo de Aprovação com Máquina de Estados

Uma solicitação percorre um ciclo de vida bem definido, que pode ser modelado como uma máquina de estados finitos: um conjunto de estados (por exemplo, pendente, aprovada e rejeitada) e de transições permitidas entre eles (**??**). Explicitar as transições válidas evita estados incoerentes: uma solicitação rejeitada não pode ser aprovada sem nova análise, e

somente uma solicitação aprovada pode ser revertida. A validação dessas regras no servidor garante que o fluxo seja respeitado independentemente da interface utilizada.

1.4.3 Snapshot e Reversão

Para possibilitar a reversão de uma alteração, é necessário preservar o estado anterior dos dados. O conceito de *snapshot* (instantâneo) consiste em capturar e armazenar o estado de um objeto em determinado momento. Ele se aproxima do padrão *Memento*, descrito por Gamma et al. (**??**), cujo propósito é capturar o estado interno de um objeto, sem violar seu encapsulamento, para que ele possa ser restaurado posteriormente.

No módulo, a entidade *Snapshot* registra o estado relevante do especialista (como categoria e pontuação) antes da aplicação de uma alteração aprovada. A reversão consiste em restaurar esses valores, mantendo a consistência entre categoria e pontuação. Cabe distinguir essa abordagem da simples exclusão do registro de histórico: a reversão é, ela mesma, uma operação registrada, de modo que o histórico permanece completo e a operação inversa pode ser auditada. Esse comportamento é análogo à ideia de transações compensatórias, em que o efeito de uma operação já efetivada é desfeito por uma nova operação, e não pela remoção do passado.

Existem abordagens mais gerais para o versionamento de dados, como o *Event Sourcing*, em que o estado é derivado de uma sequência imutável de eventos (**????**). Embora o Event Sourcing seja uma alternativa robusta, o módulo adota uma solução mais simples e proporcional ao escopo, combinando *snapshots* do estado anterior com um registro de eventos, e o conceito de *Event Sourcing* é aqui mencionado como referência conceitual para a ideia de histórico imutável.

1.4.4 Evento de Domínio e Domain-Driven Design

O Domain-Driven Design (DDD) é uma abordagem de design de software que coloca o domínio do negócio no centro do desenvolvimento, utilizando uma linguagem ubíqua para alinhar especialistas de negócio e desenvolvedores (**??**). Dentro do DDD, o conceito de Evento de Domínio (Domain Event) representa algo significativo que aconteceu no domínio do problema. Um evento de domínio é, por definição, um registro imutável de um fato passado (por exemplo, "Solicitação Aprovada"ou "Categoria Alterada"), que captura a intenção e o contexto da operação (**??**).

No contexto do Banco de Especialistas, o módulo de revisão foi modelado sob essa ótica. A entidade SolicitationReview não atua apenas como uma tabela de auditoria, mas como o registro do Evento de Domínio que materializa a decisão tomada pelo avaliador. Ao registrar o evento (a revisão) de forma imutável e com autoria definida, o sistema garante que as transições de estado do especialista (como a mudança de categoria) sejam disparadas a partir de um fato consumado e rastreável, promovendo o desacoplamento entre a intenção (solicitação) e a sua efetivação (evento de aprovação).

1.4.5 Timeline e Registro de Eventos

A *timeline* é a apresentação ordenada cronologicamente dos eventos associados a uma entidade. Cada evento (criação, aprovação, rejeição, reversão) é registrado com seu tipo, autor, data e informações complementares, e a interface os exibe como uma linha do tempo. Trata-se de uma forma de tornar a auditoria acessível ao usuário final, e não apenas ao administrador do banco de dados.

1.5 Front-end e Controle de Acesso

1.5.1 Vue.js

O Vue.js é um framework progressivo para construção de interfaces de usuário, que permite criar componentes reativos e reutilizáveis. Ele facilita a ligação entre os dados do modelo e a apresentação na interface, garantindo que alterações nos dados sejam refletidas automaticamente na visualização (**??**).

1.5.2 Nuxt

O Nuxt é um framework baseado em Vue.js que simplifica a criação de aplicações universais, com renderização no lado do servidor e geração de páginas estáticas. Ele oferece uma estrutura organizada

1.5.3 RBAC e Controle de Acesso

O controle de acesso baseado em papéis (RBAC - Role-Based Access Control) é um modelo de segurança que restringe o acesso a recursos do sistema com base nas funções atribuídas aos usuários (**??**). No módulo, diferentes papéis (como solicitante, analista e administrador) determinam as operações que cada usuário pode realizar, garantindo que apenas indivíduos autorizados possam aprovar, rejeitar ou reverter solicitações.

1.6 Verificação e Ambiente

1.6.1 Testes Automatizados com Pytest

O Pytest foi empregado para verificar as regras de negócio do módulo, em especial as transições permitidas da máquina de estados: criação, aprovação, rejeição e reversão (**??**). Os testes também verificaram os efeitos dessas transições sobre a categoria, a pontuação e o histórico da solicitação.

1.6.2 Ambiente Reprodutível com Docker

O Docker foi utilizado para reproduzir o ambiente necessário aos testes de integração, incluindo a API e o PostgreSQL, de modo que a validação dos fluxos ocorresse sob uma

configuração consistente entre execuções (**??**).

1.7 Controle de Versão e Integração Contínua

A gestão de configuração de software compreende a identificação, o controle e o registro das alterações realizadas nos artefatos de um projeto ao longo do tempo, de modo que seja possível reconstituir versões anteriores e identificar a origem de cada mudança (**??**). O controle de versão é o mecanismo que concretiza esse registro sobre o código-fonte: cada alteração é capturada em um commit, com autor, data e descrição, e o histórico resultante permite recuperar estados anteriores do sistema.

O módulo utilizou Git (**??**) para esse fim, com ramificações (branches) por funcionalidade, o que isolou o desenvolvimento do módulo das demais partes do Banco de Especialistas até que as alterações fossem verificadas. Os repositórios foram hospedados no GitLab (**??**), adotado como padrão pelo Laboratório Assert para os projetos do Polo de Inovação. A integração contínua automatiza a execução de verificações, como a suíte de testes Pytest, a cada commit enviado ao repositório; neste projeto, essa execução é realizada por uma pipeline, impedindo que alterações que quebrem as regras de negócio sejam incorporadas à base principal.

Cabe observar que o controle de versão atua sobre o código-fonte, e não sobre os dados da aplicação. A rastreabilidade das alterações cadastrais dos especialistas, tratada na seção 1.4, é um problema distinto, resolvido no domínio da aplicação por meio de solicitações, snapshots e registro de eventos.

1.8 Síntese do Capítulo

As tecnologias e os conceitos apresentados compõem a base sobre a qual o módulo de solicitações foi desenvolvido. A arquitetura cliente-servidor e o estilo REST definem a forma de comunicação entre as interfaces e a API; o Flask estrutura a aplicação, o SQLAlchemy e o PostgreSQL garantem a persistência transacional, e o Marshmallow assegura a validação e o contrato dos dados. Os conceitos de auditoria, máquina de estados, *snapshot* e registro de eventos sustentam os requisitos de rastreabilidade e reversão que motivaram o estágio. Por fim, o Pytest garante a verificação das regras de negócio, o Git organiza a colaboração e o Docker fornece o ambiente reprodutível de execução. O próximo capítulo descreve como esses elementos foram aplicados nas atividades realizadas.